

TECHNICAL DEEP DIVE

大模型推理的内存与计算解耦：AFD 架构解析与性能验证

基于技术演讲与配套材料整理

配套材料：AFD-Plugin(1).pptx、FastAFD_Vllm_Study_Slides(1).pptx

2026-08-15

原视频：[vLLM 小课堂：AFD 与 FastAFD](#) · 配套资料：[AFD Plugin 与 FastAFD 课件](#)

当大语言模型的上下文窗口从 4K 扩展到 128K，推理系统面对的不只是计算量的线性增长，更是一对深层资源矛盾的急剧放大 - Attention 对显存的饥渴与 MoE 对大批次的依赖，在同一组 GPU 上互相扼杀。本文围绕 AFD (Attention-FFN Disaggregation) 架构及其高性能实现 FastAFD，拆解这一矛盾的成因、解耦方案的工程实现，以及性能收益背后的数学本质。

适读人群： 有大模型推理工程经验、了解 KV Cache 与 MoE 基本原理的后端及 AI 系统工程师。

前置知识： 理解 Prefill 与 Decode 两阶段的区别；了解 KV Cache 如何占用显存；熟悉 MoE 模型的门控路由机制。

阅读目标：

- 理解长上下文推理中 Attention 与 MoE 的资源错配矛盾及其量化表现
- 掌握 AFD 架构的核心设计思路、角色分工与数据流闭环
- 解析双微批流水线如何在端到端层面隐藏跨节点通信延迟
- 了解 FastAFD 在 GPU 侧的底层优化手段与性能预测模型
- 客观认识当前实验阶段的局限条件与工程距离

1. 背景与瓶颈：长上下文为何会"饿死"MoE？

本节核心问题： 处理长上下文推理时，GPU 计算利用率为何出现断崖式下跌？直觉上更长的上下文意味着更多计算量，GPU 应该更忙 - 但实测却完全相反。

两类模块的正交需求

一次 Transformer 推理步骤可拆为两个核心模块：

模块	核心瓶颈	扩大吞吐的关键
Attention (自注意力)	需要读写 KV Cache (Key-Value Cache, 键值缓存)，内存占用随上下文长度线性增长	足够的显存容量
MoE FFN (Mixture of Experts, 混合专家前馈网络)	每个专家的权重矩阵很大，但单次只被少量 Token 激活	足够多的 Token 同时喂入，使专家级 GEMM 达到高吞吐

Attention 吃内存，MoE 吃批次。 两者对资源的需求方向几乎正交。

共置部署下的因果链

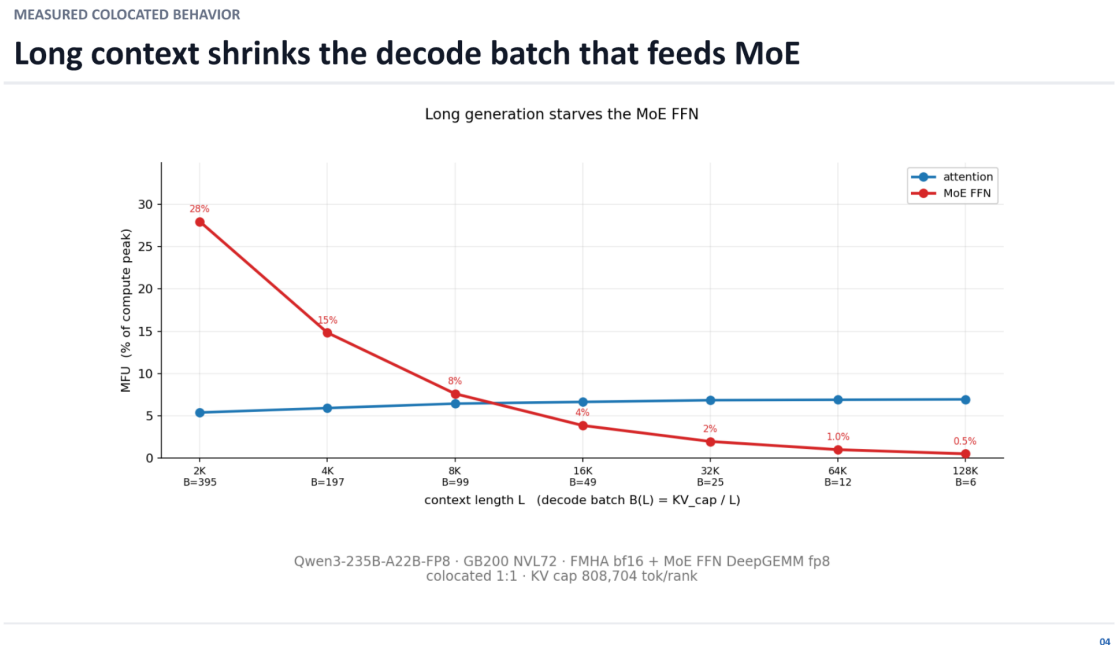
在传统共置 (Colocated) 部署中，Attention 与 MoE FFN 运行在同一组 GPU 上，显存要同时承载 KV Cache 和模型权重。当上下文长度 L 增大时：

- 1 **KV Cache 膨胀** - 每条请求的缓存占用与 L 成正比，GPU 剩余显存迅速收紧。
- 2 **并发请求数被迫缩减** - 显存总量不变，能同时驻留的 Decode 请求数 B 近似按 $B \propto 1/L$ 下降。
- 3 **MoE 专家接收的 Token 批次萎缩** - B 变小意味着每步送入 FFN 的 Token 总量锐减，专家级矩阵乘法的算术强度骤降。
- 4 **GPU 计算单元大面积闲置** - MoE FFN 本应贡献绝大部分浮点运算，此时其 MFU (Model FLOPs Utilization, 模型算力利用率) 跌至个位数甚至不到 1%。

决定 Attention 能服务多少请求的是"内存"，而决定 MoE 效率的是"请求数量"。共置部署将前者的内存饥渴直接传导为后者的计算饥饿。

实测：从 28% 到 0.5% 的坠落

为了直观展示这一传导效应有多剧烈，下图给出了共置部署下两类模块的 MFU 随上下文长度的实测变化曲线。



图注：长上下文对 Decode 批次及 MoE 利用率的挤压效应。测试条件：Qwen3-235B-A22B-FP8 模型，GB200 NVL72 (NVIDIA 机架级系统) 平台，Attention 使用 bf16 FMHA、MoE FFN 使用 fp8 DeepGEMM，每 rank KV Cache 上限 808 704 Token。来源：演讲 PPT，第 20 页。

图中关键元素解读：

- **横轴**为上下文长度 L (从 2K 到 128K) ，括号内附对应的 Decode 批次大小 B。
- **纵轴**为 MFU，即实际浮点吞吐占硬件峰值的百分比。
- **蓝色线 (Attention)**：始终平稳徘徊在约 5% - 7%，对上下文长度不敏感 - Attention 本身是 memory-bound 操作，利用率由访存带宽决定。
- **红色线 (MoE FFN)**：在 L = 2K、B = 395 时达到 28%；L 翻倍到 4K、B 腰斩至 197 时降到约 15%；此后一路滑坡 - L = 32K 时约 2%，L = 128K 时跌至 0.5%，相较峰值**缩水 56 倍**。

状态演进对照

上下文长度	Decode 批次 B	MoE FFN MFU	Attention MFU	状态描述
2K	395	28%	≈ 5%	MoE 效率尚可，GPU 计算单元有事可做
128K	6	0.5%	≈ 5%	MoE 几乎空转，绝大部分算力被浪费

从 395 条请求缩减到仅 6 条 - 不是调度策略失误，而是物理显存无法同时容纳更多 128K 请求的 KV Cache。

结论：共置部署把 Attention 的内存瓶颈传导成了 MoE 的计算瓶颈，上下文越长传导越剧烈。需要注意，上述数据基于 Qwen3-235B-A22B 模型与 GB200 NVL72 硬件。对于参数量更小或专家数更少的 Dense 模型，MoE FFN 占比低，矛盾未必突出。但在主流 MoE 大模型持续走向更长上下文窗口的趋势下，这一结构性错配已成为部署效率的首要障碍。

明确了共置部署导致的资源错配后，接下来的问题是：能否从物理架构层面打破这一枷锁？

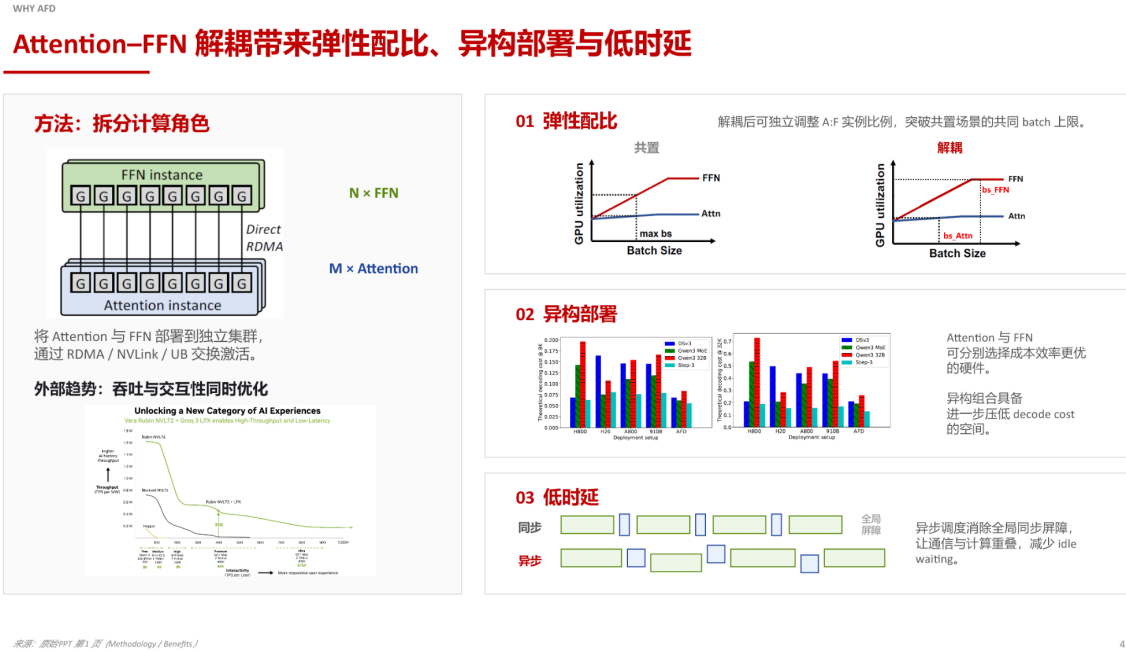
2. 核心设计：AFD 架构的弹性配比与异构部署

本节核心问题：如何在物理层面解除 Attention 和 MoE 之间的资源绑定，让两类算子各自运行在最高效的工作点上？

一句话思路

AFD (Attention-FFN Disaggregation, Attention-FFN 解耦) 的做法直截了当 - 把 Attention 和 FFN 从同一块 GPU 上拆开，分别部署到两个独立集群，再用高速互联交换中间激活值。

下图展示了 AFD 的系统全景与三项核心收益，为后续各节的展开提供整体框架。



图注：AFD 解耦后的系统概览与三项核心收益。来源：演讲 PPT，第 3 页。

图左侧展示了拆分后的拓扑：**M 个 Attention 实例**与**N 个 FFN 实例**各占一组独立节点，二者之间通过**RDMA、NVLink 或 UB (Unified Buffer)** 交换激活张量。图右侧依次展开三项直接收益。

收益一：弹性配比 - 解除 Batch Size 的公共上限

维度	共置模式	AFD 解耦模式
Batch Size 决定因素	Attention 与 FFN 共享显存，取二者可容纳的较小值	各自独立，Attention 按 KV Cache 容量定 Batch，FFN 按算力最优点定 Batch
A:F 实例比例	固定 1:1	可按实际负载调整为 M:N
GPU 利用率	一侧受限时另一侧空转	两侧各自逼近峰值利用率

因果链如下：共置时 KV Cache 占满显存，限制并发请求数 → 并发上限同时约束了 FFN 获得的 Batch Size。解耦后，多个 Attention 节点可以在各自完成 Self-Attention 后，将产生的激活值**汇聚**到同一个 FFN 节点。假设单个 Attention 节点受显存限制只能承载 128 条并发请求，4 个 Attention 节点汇聚后 FFN 看到的有效 Batch Size 即为 512 - MoE 的大批次计算效率由此恢复。

边界条件：弹性配比的上限受限于 Attention-FFN 之间的网络带宽。当激活值传输时间超过计算时间时，增加 Attention 节点不再带来收益。

收益二：异构部署 - 为不同算子选择最匹配的硬件

Attention 是**内存密集型**任务，理想硬件应有大 HBM 容量和高内存带宽；FFN / MoE 是**计算密集型**任务，理想硬件应有高 FP8/FP16 算力。共置时两类算子必须运行在同型号的加速卡上，无法同时满足两种需求。解耦后，Attention 集群可选择内存容量更大的芯片，FFN 集群可选择算力密度更高的芯片。PPT 第 3 页的柱状图以不同硬件组合展示了理论解码成本的下降趋势（因柱状图细节较小，具体数值仅作参考）。这种异构组合在硬件代际更替时尤为有价值 - 无需整体替换，只需升级瓶颈侧的集群。

收益三：异步调度 - 消除全局同步屏障

共置的同步模式中，每层推理完成后需全局屏障，快节点等慢节点。AFD 天然允许流水线化：Attention 完成当前层后立即发送激活并开始处理下一批次，FFN 收到后立刻计算并回传，通信与计算可以重叠执行。这一特性将在第 4 节结合具体的流水线机制详细展开。

小结： AFD 的核心因果链是**物理拆分** → **独立扩缩** → **各自高效**。但架构解耦也引入了新的工程挑战：激活值的高频跨节点传输对网络提出苛刻要求，集群内部的控制流与数据流也需要重新设计 - 这正是下一节要拆解的内容。

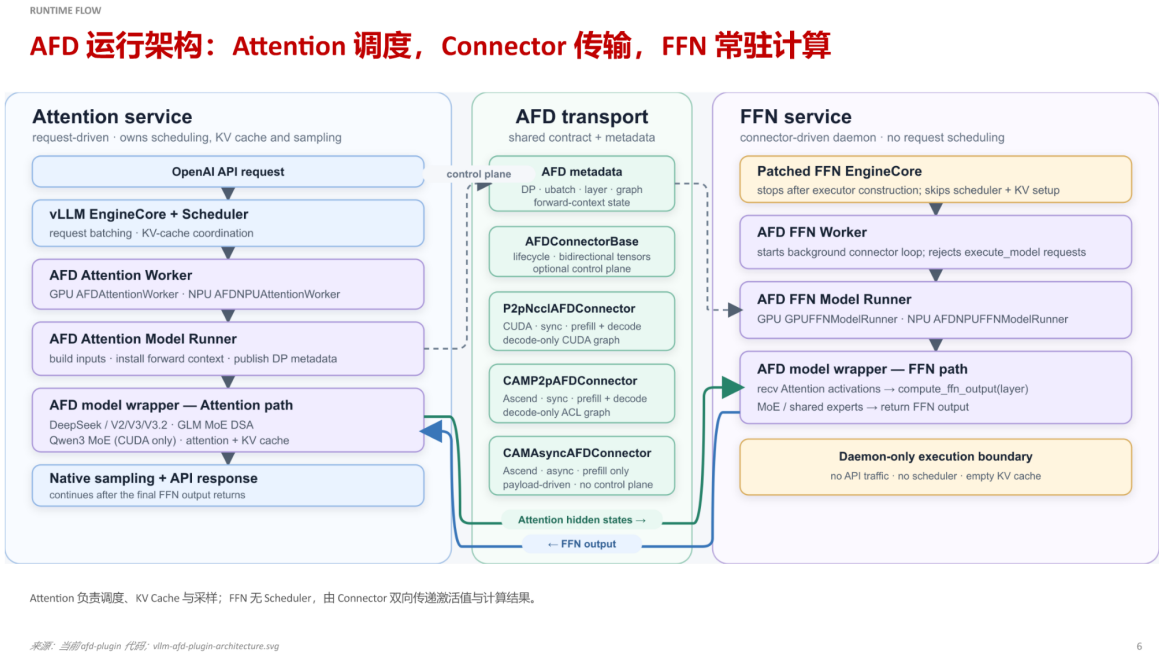
3. 调度与计算的边界重塑：解耦后的角色分工

本节核心问题： Attention 节点和 FFN 节点各自承担什么角色？控制流和数据流如何完成闭环？

不对称的两类节点

传统推理引擎中，每个节点既是调度者又是计算者。AFD 沿 Transformer 层的天然边界将两种职能一分为二，形成完全不对称的角色分配。

下图展示了解耦后的运行时架构，明确了三个区域 - Attention service、AFD transport、FFN service - 的边界与交互方式。



图注：AFD 运行架构。左侧 Attention service 持有调度器、KV Cache 与采样逻辑；右侧 FFN service 退化为无调度器的守护进程；中间 AFD transport 负责控制面与数据面的双向传输。来源：演讲 PPT，第 6 页。

Attention service：系统的控制中枢

职能	说明
API 入口	接收并解析外部推理请求，是唯一面向用户的入口
请求调度 (Scheduler)	决定哪些请求参与当前迭代、如何组批
KV Cache 管理	分配与回收键值缓存空间，维护每条请求的上下文状态
注意力计算	执行 Self-Attention 前向传播
采样 (Sampling)	根据 logits 完成 Token 采样并返回结果

关键约束：**请求状态 (Request state) 始终留在 Attention 侧**。调度器掌握每条请求处于 Prefill 还是 Decode 阶段、已使用多少 KV Cache 空间、是否超时等全部元信息，这些状态不跨节点传递。

FFN service：纯粹的计算守护进程

FFN service 被剥离了所有管理功能，退化为一个由 Connector 驱动的守护进程 (Daemon，即常驻后台进程)：

- 没有 API 流量 - 外部请求不会到达 FFN 节点
- 没有 Scheduler - 不做任何请求级调度决策
- KV Cache 为空 - 键值缓存完全由 Attention 侧持有
- 唯一动作**：接收激活值 → 执行前馈/专家计算 → 返回结果

这种 **Daemon** 化设计使得 **FFN** 节点可以独立扩缩容，不需要同步任何请求级状态，大幅降低了集群编排复杂度。

数据流闭环：一次 Decode 迭代的完整路径

- 1 **Attention 侧计算**：Scheduler 组批后，执行 Self-Attention，产出当前层的隐藏状态（Hidden states）。
- 2 **发送激活值**：通过 Connector 数据面将隐藏状态发送至 FFN 节点，控制面同步传递批次维度等元数据。
- 3 **FFN 侧计算**：Daemon 接收激活后执行前馈网络计算（MoE 模型即门控路由 + 专家计算），产出该层输出。
- 4 **返回结果**：FFN 将计算结果通过 Connector 回传 Attention service。
- 5 **继续后续层**：Attention 拿到 FFN 输出后进入下一层注意力计算，或在最后一层执行采样返回 Token。

整个闭环中，**只有路由后的隐藏状态在网络中传输**，请求完整上下文、KV 数据和调度元信息均不跨越节点边界。 **Cache**

Connector 的控制面与数据面分离

AFD transport 内部分为两个通道：**控制面**传递 **AFD metadata** 等协调信息，确保两侧对当前批次形状和含义达成一致；**数据面**承载实际激活张量传输，根据硬件拓扑可选择 **P2pNccl**、**CAMP2p**、**CAMAsync** 等不同 Connector 实现。二者分离意味着底层传输协议更换时，上层调度与 FFN 执行逻辑均无需修改。

注意：演讲材料未给出 FFN Daemon 的线程模型和激活值缓冲区管理的具体实现细节，这些需结合开源代码进一步核验。

小结：解耦将调度权和数据所有权完全收归 **Attention** 节点，FFN 退化为无状态的计算单元。两侧可独立演进，但每次迭代必须经历一轮跨节点的激活值往返 - 这一网络传输开销如何被消化，是接下来流水线设计要解决的核心问题。

4. 双微批流水线：隐藏跨节点通信延迟

本节核心问题：每层推理至少需要两次 **P2P** 通信（Attention→FFN 和 FFN→Attention），这些额外的网络往返会不会抵消解耦带来的计算收益？

结论是：**不会**。**AFD** 通过 **DBO**（Double Batch Overlap，双微批重叠）机制，将通信过程嵌入计算窗口，使网络延迟几乎被完全掩盖。

核心思路：一个 Batch 拆成两个微批

DBO 的设计只有一步 - 将完整的 Decode 批次拆为两个大小相等的微批次（Microbatch，即 uBatch1 和 uBatch2），让它们在 Attention 侧和 FFN 侧**错开执行**。当 Attention 正在处理 uBatch2 时，FFN 侧同时处理 uBatch1 并完成通信返回，两段工作在时间上完全重叠。实现这一重叠的组件是 CAMP2PConnector，它负责协调 Attention↔FFN 之间的 P2P 通信和微批调度。

流水线时序

下图展示了 CAMP2PConnector 的双微批重叠工作方式，是理解 DBO 如何消除通信延迟的关键。



DBO 将通信嵌入双微批流水，Attention 的执行窗口完整覆盖当前批 FFN
来源: [q4 plugin/docs/hpu/dbo.png](#); 流水图源 imageGen 在流图语义上定向构造

图注：CAMP2PConnector 双微批完全重叠示意图。来源：演讲 PPT，第 8 页。

横轴为时间，纵轴分为 Attention 侧和 FFN 侧两条泳道。四个阶段的运转如下：

阶段	Attention 侧	FFN 侧	通信方向
1) F2A 接收 + Attention 计算	接收上一层 FFN 返回的激活，执行 uBatch1 的 Attention	空闲或处理上一轮残余	FFN → Attention
2) A2F 发送 + 启动下一微批	uBatch1 完成后立即发往 FFN； 同时 启动 uBatch2 的 Attention	开始接收 uBatch1	Attention → FFN
3) FFN 被覆盖	继续执行 uBatch2 的 Attention	完整计算 FFN(uBatch1)	-
4) F2A 返回 + 进入下一轮	uBatch2 完成；接收 FFN(uBatch1) 的输出	将结果发回 Attention	FFN → Attention

图中最关键的区域标注为 **Overlap Window**：Attention(uBatch2) 的执行窗口从 2) 延伸到 3) 结束，而 FFN(uBatch1) 的全部计算和通信恰好落在这个窗口之内。**FFN 的时延被 Attention 的计算完整覆盖，不在关键路径上产生额外等待。**

最小状态演进

假设系统当前处于第 l 层 Decode 阶段：

- 1 **t₀**：Attention 侧收到第 l-1 层 FFN 的输出，开始计算 Attention(uBatch1)。
- 2 **t₁**：Attention(uBatch1) 完成，结果通过 P2P 发往 FFN。Attention 侧**不等待** FFN 返回，立即开始 Attention(uBatch2)。
- 3 **t₁ ~ t₂**：FFN 收到 uBatch1 激活后执行专家计算。同一时段 Attention 侧在处理 uBatch2。
- 4 **t₂**：Attention(uBatch2) 完成，FFN(uBatch1) 也已完成并返回。进入第 l+1 层.....

两个微批交替推进，FFN 的计算与通信始终被下一个微批的 Attention 计算遮蔽。

因果链

切分微批次 → 错开执行窗口 → 掩盖通信开销

切分使 Attention 获得两段独立的计算单元；错开使 uBatch2 的 Attention 在 uBatch1 发往 FFN 后立即启动；掩盖使 FFN 的全部时间被 Attention 执行窗口包含。

边界条件

DBO 完全隐藏 FFN 延迟的前提是：

$T_{Attn}(uBatch2) \geq T_{FFN}(uBatch1) + T_{comm}$

- $T_{Attn}(uBatch2)$ ：Attention 处理第二个微批的计算耗时
- $T_{FFN}(uBatch1)$ ：FFN 处理第一个微批的耗时
- T_{comm} ：A2F + F2A 双向通信总时间

在长序列解码或 KV Cache 较大的场景下，Attention 计算量充裕，不等式容易满足。反之，若序列极短导致 Attention 极快，FFN 的计算与通信可能无法被完全覆盖，流水线会出现气泡（Bubble）。此外，FFN 侧经过解耦后仅承担纯计算（不再管理 KV Cache），其执行时间本身已被压缩，这进一步放宽了不等式约束。

流水线机制在宏观上隐藏了延迟，但在极高性能的 GPU 硬件上，微观的内核调度开销依然致命。下一节将探讨 FastAFD 如何从运行时层面逐一消灭这些隐性损耗。

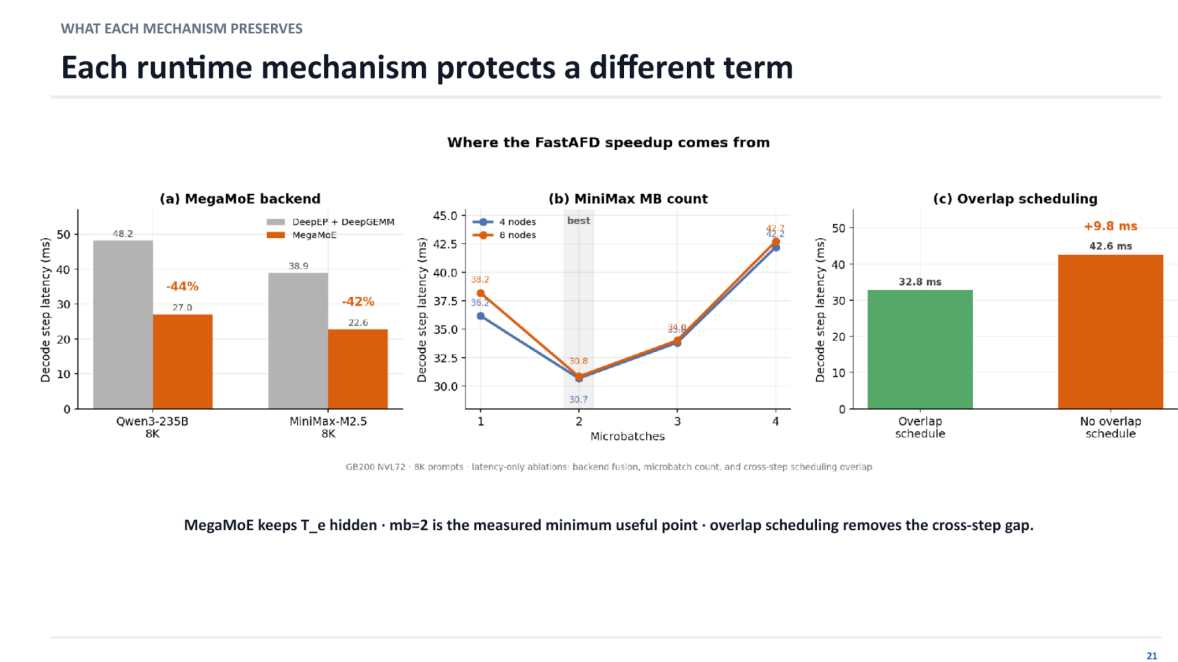
5. 底层优化：FastAFD 的算子融合与微批次调优

本节核心问题：宏观的架构解耦和流水线重叠之外，高性能硬件上的微观调度开销 - 内核启动、显存缓冲、步间空隙 - 会不会吞噬理论加速比？

FastAFD 在 GB200 NVL72（NVIDIA 机架级系统）上针对三类隐性开销分别给出了运行时优化。下面结合消融实验逐一展开。

消融实验总览

下图包含三组子图，分别对应后续三个小节。纵轴均为单步解码延迟（ms），越低越好。



图注：FastAFD 消融实验，测试硬件 GB200 NVL72，prompt 长度 8K。来源：演讲 PPT，第 37 页。

优化一：MegaMoE 算子融合

传统 MoE 推理将 Token 路由/移动与专家 GEMM 拆为独立内核，每次启动都有 GPU 调度开销，中间结果还需暂存于显存缓冲区。MegaMoE 将二者融合为单一算子，Token 数据在寄存器或共享内存中直接流向 GEMM 计算，不再落地到全局显存。

消融数据（图 a，对比分离算子 "DeepEP + DeepGEMM" 与 MegaMoE 融合算子）：

模型	分离算子延迟	MegaMoE 延迟	降幅
Qwen3-235B	48.2 ms	27.0 ms	-44%
MiniMax-M2.5	38.9 ms	22.6 ms	-42%

以上数据在 GB200 NVL72、8K prompt 条件下测得。

因果链：分离算子 → 额外缓冲 + 多次启动 → 延迟高；融合算子 → 缓冲消除 + 启动减少 → 延迟直降四成以上。

优化二：微批次数的最优选择

流水线并行的经典做法是将切成若干微批次以实现计算-通信重叠。微批次越多，重叠覆盖率理论上越高但切分本身引入调度与同步成本。

Batch

-

图 b 横轴为微批次数（1 - 4），关键读数：

微批次数	4 节点延迟 (ms)	8 节点延迟 (ms)
1	38.2	34.8
2	36.2	33.9
3	30.8	43.3
4	30.7	42.2

4 节点配置下延迟随微批次增加持续下降但边际递减；8 节点配置下**微批次数 = 2 时达到最低点**，继续增加反而因节点间同步开销导致延迟大幅回升。用一个简化模型解释：单步延迟近似为 $L(m) = C/m + m \cdot d + \text{sync}(m)$ ，其中 C/m 是每个微批次的有效计算时间、 $m \cdot d$ 是启动开销、 $\text{sync}(m)$ 是同步成本。 $m=2$ 时计算收益与额外开销取得实测最佳平衡，因此 FastAFD 将 **2 个微批次**作为默认配置。

优化三：重叠调度消除跨步间隙

即使单步内部计算与通信已良好重叠，相邻 Decode 步之间仍有一段"真空" - 上一步结果写回后，下一步调度才启动。重叠调度（Overlap Scheduling）在当前步尾部计算未结束时，提前启动下一步的路由决策和 Token 预取。

消融结果（图 c）：

- 无重叠调度：42.6 ms
- 启用重叠调度：32.8 ms
- 单步节省 9.8 ms**，约占总延迟的 23%

三层优化的协同关系

机制	消除的开销	典型收益
MegaMoE 算子融合	内核启动 + 显存缓冲	延迟 -42% ~ -44%
微批次 = 2	流水线空泡 vs 调度开销的平衡	达到实测延迟最低点
重叠调度	跨步间隙	单步 -9.8 ms

三者缺一不可：算子融合压缩单次计算的绝对耗时，微批次调优在重叠与切分开销间找平衡点，重叠调度填补步间最后的空白。这些底层工程是将理论加速比兑现为实际吞吐的必要条件。

边界条件：所有数据均在 GB200 NVL72、8K prompt 下测得。不同硬件拓扑或不同 prompt 长度下，最优微批次数与各项收益比例可能变化。

6. 性能与数学模型：加速比的来源与预测

本节核心问题：

FastAFD

能带来多少真实的吞吐量提升？加速比的数学本质是什么？能否事前准确预测？

实测加速幅度

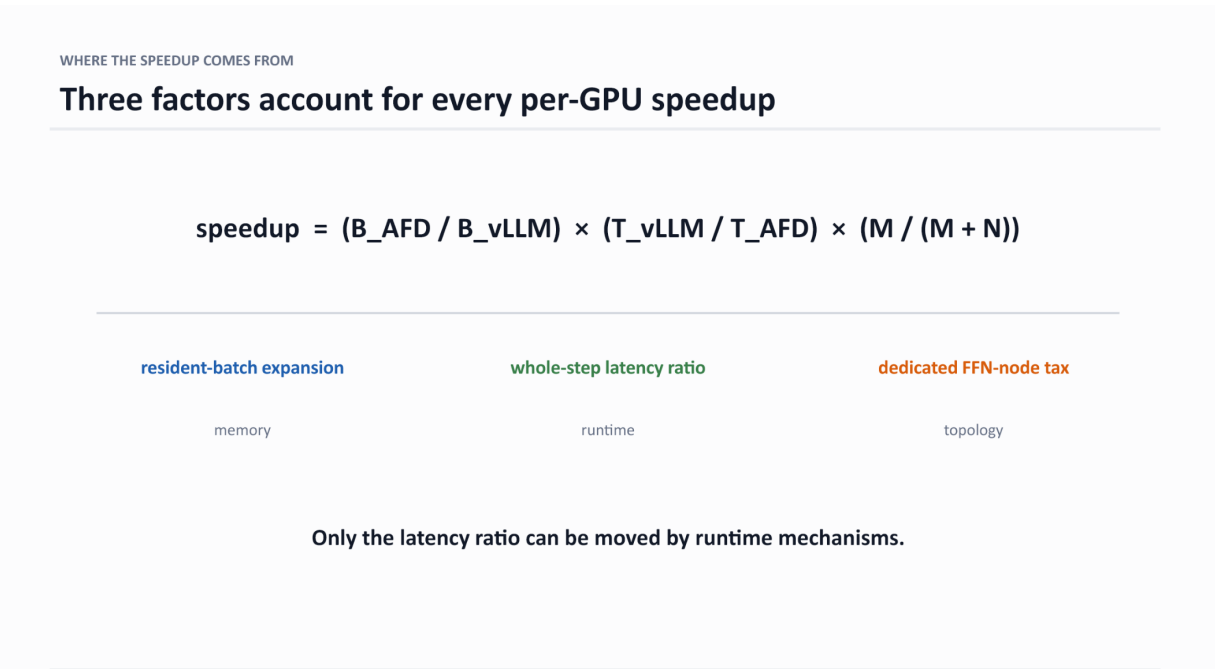
在 GB200 NVL72 上, 以 Qwen3-235B-A22B-FP8 和 MiniMax-M2.5 两款 MoE 模型为对象, 输入提示词长度分别为 8K 和 16K Token 的稳态解码场景下, FastAFD 相较于 vLLM 基线实现了 **1.35 - 1.45 倍的单卡解码吞吐量提升**。衡量解码吞吐的核心指标是 TPOT (Time Per Output Token, 每个输出 Token 的生成时延)：TPOT 越低, 单卡每秒可完成的解码 Token 数越多, 吞吐量越高。

条件维度	具体约束
硬件平台	NVIDIA GB200 NVL72
测试模型	Qwen3-235B-A22B-FP8、MiniMax-M2.5 (均为大规模 MoE)
测试阶段	稳态 Decode (系统已处于满载解码状态)

脱离上述条件谈加速比并无意义。

加速比的三因子分解

1.35 - 1.45× 的 TPOT 改善究竟从哪里来？演讲材料给出了将单卡加速比拆解为三个独立因子乘积的公式。下图展示了该公式的结构。



图注：FastAFD 单卡加速比的数学分解，展示内存、运行时与拓扑三个维度的贡献因子。来源：演讲 PPT，第 33 页。

$$\text{speedup} = (B_AFD / B_vLLM) \times (T_vLLM / T_AFD) \times M / (M + N)$$

因子一：常驻请求扩展（内存维度）

B_AFD 是解耦后 Attention 节点的常驻请求数，B_vLLM 是基线下同等 GPU 显存条件下的值。解耦后 Attention 节点不再存放 FFN 权重，释放的显存可容纳更多 KV Cache 条目，承载更多并发请求。**该因子约为 1.5 倍** - 这是加速的最大单一来源，本质上是用拓扑换显存、用显存换并发。

因子二：全步延迟比（运行时维度）

T_vLLM 是 vLLM 基线完成一个完整 Decode 步的 TPOT，T_AFD 是 FastAFD 的等效 TPOT。DBO 流水线将 FFN 计算与 Attention 在时间上重叠，使 T_AFD 接近单独 Attention 步骤的延迟，因此比值 > 1。演讲材料指出，三个因子中**只有延迟比可以被运行时机制持续调节** - 常驻请求扩展由模型结构和显存容量决定，FFN 节点税由拓扑决定，唯独延迟比能通过 DBO 等调度策略优化。

因子三：专用 FFN 节点税（拓扑维度）

M 是 Attention 的 GPU 数，N 是专用 FFN 的额外 GPU 数。M/(M+N) 始终 < 1，是引入额外硬件的折扣因子。N 越大，FFN 并行度越高（有利于因子二），但节点税也越重（不利于因子三）。最优 M:N 比例取决于模型的 Attention/FFN 计算比和网络带宽。

三因子协同：定性示例

因子	含义	示例值
B_AFD / B_vLLM	常驻请求扩展	1.50
T_vLLM / T_AFD	全步延迟比	1.15
M / (M+N)	FFN 节点税	0.85

$$\text{speedup} = 1.50 \times 1.15 \times 0.85 \approx 1.47$$

内存收益 (1.5×) 已足够强劲；流水线带来 15% 的延迟改善；而拓扑代价 (0.85) 将理论上限 1.50 × 1.15 = 1.725 拉回到 1.47 - 与实测的 1.35 - 1.45 倍区间吻合。

注意：上述数字为辅助理解的定性示例，并非演讲材料中给出的精确分项数值。

预测模型的精度验证

三因子公式不仅可事后解释，还可**事前预测**。通过假设 FFN 延迟被流水线完全隐藏 (T_AFD 近似等于纯 Attention 步骤的 TPOT)，可直接利用 vLLM 的单步 profiling 数据预估解耦后的解码周期。实际验证表明，该预测方法的误差在 **2.2% 以内**。

这意味着工程师在决定是否为某个模型部署 AFD 时，无需搭建完整解耦集群，只需在标准 vLLM 环境下做一次性能采样即可高置信度预判收益，显著降低架构决策的试错成本。

小结： FastAFD 的加速并非单一优化的产物，而是内存释放 ($\approx 1.5\times$)、计算重叠 (> 1) 与硬件开销 (< 1) 三者博弈的结果。内存收益是主导项，运行时优化是唯一可持续改进的旋钮，拓扑代价是必须计入的固定成本。

7. 全阶段表现与工程现状

本节核心问题： 稳态 Decode 之外，AFD 在 Prefill 阶段和同步 Decode 阶段表现如何？走向生产就绪还面临哪些局限？

异步 Prefill：消除全局同步屏障

在传统数据并行 + 流水线并行 (DP + PCP) 方案中，Prefill 与 Decode 共享同一组卡，每次 Decode 微批次的全局同步都会阻塞正在进行的 Prefill 计算，请求并发越高 TTFT (Time To First Token, 首 Token 时延) 越差。

AFD 将 Attention 与 FFN 拆分后，Prefill 可在 Attention 节点上独立于 Decode 批次执行，不再受 FFN 侧同步屏障阻塞：

解耦 $\rightarrow$ Prefill 在 Attention 节点异步发射 $\rightarrow$ 消除全局同步等待 $\rightarrow$ TTFT 下降 $\rightarrow$ 在同等 SLO 下可接受更多请求 $\rightarrow$ 有效吞吐上升。

下图对比了异步 Prefill 方案与基线方案在 TTFT 和有效吞吐 (Goodput) 上的差异，是评估 AFD 在 Prefill 阶段实际收益的关键依据。

ASYNC PREFILL RESULTS

异步 AFD：P50 TTFT 降低 60%+，有效吞吐最高提升 2.2 $\times$

模型：DeepSeek-V3.2

对比：DP4PCP8 vs. AFD (DP3PCP3 + EP8)

硬件：Ascend 910C



异步 AFD 同时改善尾延迟与 SLO 有效吞吐

测试设置：版本 v0.19.1rc1；使用减层模型，并开启专家强制均衡 (forced expert balancing)

来源：演讲 PPT，第 4 页；DeepSeek-V3.2 / Ascend 910C

12

图注：异步 Prefill 性能对比。模型 DeepSeek-V3.2，硬件 Ascend 910C (华为昇腾 AI 处理器)，使用减层模型并开启专家强制均衡。来源：演讲 PPT，第 12 页。

图中关键元素：

元素	含义
DP4PCP8	基线：4 路数据并行 × 8 级流水线并行
AFD (DP3PCP3 + EP8)	实验方案：3 路 DP × 3 级 PCP 的 Attention 节点 + 8 路专家并行的 FFN 节点
左侧分位图	从 Mean 到 P99 逐分位展示 TTFT，AFD 在所有分位上均低于基线
右侧 Goodput 图	以 SLO 为横轴，展示满足 TTFT 约束的有效请求吞吐量

核心数据点（均在上述测试条件下取得）：

- 中负载**（请求到达速率 RQS 6 - 10）：AFD 的 **P50 TTFT 降低 60% - 70%**
 - 高负载**（RQS > 14）：P50 TTFT 仍降低约 25%，并发压力增大后收益递减但依然显著
 - 有效吞吐**：TTFT SLO 设为 2s 时 AFD 达到 0.223 req/s/die、基线仅 0.095 req/s/die，提升约 **2.2 倍**；SLO 放宽到 5s 后差距缩小至 0.272 vs. 0.218 req/s/die
- 一条直观规律：**SLO 越严格，AFD 的有效吞吐优势越明显** - 严格 SLO 下基线方案大量请求因超时被丢弃，而 AFD 的异步调度使更多请求在时限内完成。

同步 Decode：小幅但稳定的吞吐增益

除异步 Prefill 外，AFD Plugin 在同步 Decode 阶段也展现出正向收益。在以下受控条件下：

条件维度	具体约束
模型	DeepSeek-V3.2 W8A8（权重 8-bit、激活 8-bit 量化）
硬件	Ascend 910C
输出长度	512 - 1536 Token
MoE 路由策略	开启专家强制均衡
A:F 配比	64 个 Attention 实例：16 个 FFN 实例（64A16F）

64A16F 配比使同步 Decode 阶段的吞吐提升了 **9% - 11%**。相比异步 Prefill 的数倍级增益，这一幅度看似有限，但它验证了解耦架构在纯 Decode 负载下同样有效 - 即便不引入异步调度，仅靠弹性配比释放的显存和恢复的批次规模，也能带来稳定的吞吐改善。

必须正视的边界条件

每一项数据都附带严格限定，脱离这些条件的推广是不严谨的：

- 减层模型**：异步 Prefill 测试未使用 DeepSeek-V3.2 的完整层数，而是采用减层版本。减层降低了单节点计算与通信压力，全量模型的收益可能不同。
- 专家强制均衡**：MoE 路由天然存在负载不均，上述测试中均开启了强制均衡策略使每个专家处理等量 Token。真实推理关闭此选项后，FFN 节点间的负载倾斜可能抵消部分增益。
- 硬件绑定**：异步 Prefill 和同步 Decode 的数据均在 Ascend 910C 上取得，未给出同配置在 GB200 NVL72 或其他硬件上的对比。

- 4 **软件成熟度**：测试基于 vLLM v0.19.1rc1（候选发布版本），AFD Plugin 在社区仓库中仍处于 experimental 状态，Open PR 不等同于生产就绪。
- 5 **动态负载适应性**：测试采用固定请求到达速率，未涉及突发流量或请求长度剧烈波动等真实在线场景，AFD 在动态负载下的稳定性仍是开放问题。

结论与展望

核心结论

- 1 **长上下文推理放大了 Attention 与 MoE 的资源错配**。上下文从 2K 增长到 128K 时，Decode 批次从 395 缩减至 6，MoE FFN 的 MFU 从 28% 跌至 0.5%（测试条件：Qwen3-235B-A22B-FP8，GB200 NVL72）。共置部署将内存瓶颈直接传导为计算瓶颈。
- 2 **AFD 通过物理隔离实现弹性配比**。将 Attention 与 FFN 部署到独立集群后，多个 Attention 节点可汇聚 Token 喂给 FFN 节点，恢复 MoE 所需的大批次计算效率；同时允许为两类算子选择最匹配的异构硬件。
- 3 **Attention 侧保留全部调度权，FFN 侧退化为无状态 Daemon**。只有隐藏状态跨节点传输，请求元信息和 KV Cache 不越界，这一不对称设计降低了集群编排复杂度。
- 4 **DBO 双微批流水线是隐藏通信延迟的关键**。将 Batch 拆为两个微批次后，Attention(uBatch2) 的执行窗口完整覆盖 FFN(uBatch1) 的计算与通信时间，使跨节点往返不出现在关键路径上。
- 5 **底层运行时优化将理论收益转化为实际吞吐**。MegaMoE 算子融合使单步延迟降低 42% - 44%；2 个微批次在流水线重叠与调度开销间取得最佳平衡；重叠调度消除跨步间隙，单步节省 9.8 ms（均为 GB200 NVL72、8K prompt 条件下测得）。
- 6 **加速比由内存、运行时和拓扑三因子共同决定**。常驻请求扩展 ($\approx 1.5\times$) 是主导贡献源，全步 TPOT 延迟比是唯一可持续优化的旋钮，FFN 节点税是必须计入的固定成本。三因子乘积在 GB200 NVL72 上实测为 1.35 - 1.45 倍单卡解码吞吐提升，性能预测模型误差在 2.2% 以内。
- 7 **解耦在 Prefill 与 Decode 阶段均有正向收益**。异步 Prefill 在 Ascend 910C、DeepSeek-V3.2 减层模型并开启专家强制均衡的条件下，中负载 P50 TTFT 降低 60% - 70%，有效吞吐最高提升 2.2 倍；同步 Decode 在同一硬件和模型上，64A16F 配比带来 9% - 11% 的吞吐改善。

现有局限

- 所有性能数据均依赖于特定硬件（GB200 NVL72 或 Ascend 910C）、特定模型和受控测试配置，跨平台与跨模型的泛化性尚待验证。
- AFD Plugin 目前仍为 experimental 状态，Open PR 不代表正式支持或生产就绪。
- 测试中使用了减层模型和专家强制均衡等简化条件，全量模型在动态负载和自然路由分布下的表现仍是开放问题。
- FastAFD 的性能预测公式在 Hybrid 推理（Prefill 与 Decode 混合）状态下可能不再适用。